

CoreOps Week 5

Objective: Understand how Kubernetes workloads behave during failures, how to design graceful shutdown, how backups & restore workflows operate, and how to prepare clusters for multi-zone resilience.

MONDAY: PDBs, preStop Hooks & Graceful Shutdown

Critical Concepts

- What happens when a pod is terminated
- SIGTERM → graceful shutdown → SIGKILL timeline
- How **readiness probes** protect traffic during shutdown
- **PodDisruptionBudget (PDB)**: guaranteeing app availability
- **preStop hooks**: draining connections before shutdown

Key Outcomes

- You understand why pods may be killed abruptly
- You can design zero-downtime rollouts
- You know how to protect a Stateful application from voluntary disruptions

WHAT TO PRACTICE

Lab 1: Apply a PodDisruptionBudget: Ensure at least 1 pod stays available during node drain.

Lab 2: Add a preStop Hook: Gracefully drain traffic before shutting down the container.

Lab 3: Verify graceful termination sequence: Trigger termination → watch logs → observe probe behavior.

WHERE TO STUDY

- Kubernetes Docs → PDBs
- Probes & lifecycle hooks
- SIGTERM handling in containers

INFRAThrone MINDSET NOTES

- PDB protects *availability*, not *performance*.
- Graceful shutdown is part of **SRE golden signals**.

TUESDAY: Backup & Restore: Velero + VolumeSnapshot

Critical Concepts

- Velero backup architecture
- PVC backups vs volume snapshots
- Cluster state backup
- Restoring namespaces & workloads
- Cross-region / cross-cluster recovery

Key Outcomes

- You know how to back up workloads
- You understand when snapshots vs Velero backups are appropriate
- You can restore workloads after accidental deletion

WHAT TO PRACTICE

Lab 1: Install Velero: Use local/minio backend or cloud provider.

Lab 2: Take a namespace backup: Include deployments, services, PVCs.

Lab 3: Delete the namespace → restore it: Verify that data returns.

Lab 4: Create a VolumeSnapshot: Restore the PVC individually.

WHERE TO STUDY

- Velero official documentation
- CSI VolumeSnapshot APIs
- Cloud provider snapshot guides

INFRAThrone MINDSET NOTES

- Backup ≠ DR. DR includes - restore time, restore correctness, dependency recovery.
- Snapshots are fast but **not application-consistent** by default.
- Always test recovery... **not recovering is worse than no backup at all.**

WEDNESDAY: Chaos Engineering Fundamentals

Critical Concepts

- What Chaos Engineering *is* and *is not*
- Failure hypothesis
- Kubernetes chaos tooling
- Pod kill, node kill, network latency
- Observability validation during chaos

Key Outcomes

- You know how to inject safe failures
- You understand resilience patterns
- You can validate system behavior under stress

WHAT TO PRACTICE

Lab 1: Pod Kill Experiment: Delete random pods → observe rollout & recovery.

Lab 2: Network Latency Injection (controlled): Add delay → observe application timeout behavior.

Lab 3: Validate recovery signals: Check readiness → logs → scaling behavior.

WHERE TO STUDY

- Chaos Mesh basics
- LitmusChaos workflows
- Principles of Chaos Engineering (Gremlin guide)

INFRATHrone MINDSET NOTES

- Chaos must be **safe, predictable, observable**.
- Start with small blast radius → expand slowly.
- Resilience is built by breaking things **before production breaks you**.

THURSDAY: Multi-Zone Kubernetes Architecture

Critical Concepts

- Zonal vs Regional clusters
- Node spread, zone spread constraints
- Multi-zone autoscaling
- Control plane high availability
- Designing apps for zone outage

Key Outcomes

- You understand how workloads spread across zones
- You know the HA vs non-HA control plane differences
- You can explain the multi-zone failure tolerance design

WHAT TO PRACTICE

Lab 1: View zone topology: Check node labels & zone distribution.

Lab 2: Apply topology spread constraints: Force workload to spread evenly across zones.

Lab 3: Drain an entire zone: observe how deployments react.

WHERE TO STUDY

- GKE / AKS multi-zone documentation
- Kubernetes topology spread constraints
- High availability architecture design references

INFRATHrone MINDSET NOTES

- Zonal failures are common and expensive; design for it.
- Use topology constraints → avoid 100% pods landing in one zone.
- HA is not optional for critical infrastructure.

FRIDAY: Runbooks, Escalation Chains & Failover Drills

Critical Concepts

- What a runbook is and why SREs rely on it
- Creating actionable, step-by-step procedures
- Defining escalation chains
- Performing failover drills (controlled)
- RTO vs RPO concepts

Key Outcomes

- You can write a complete runbook
- You understand how to escalate incidents properly
- You can orchestrate a small DR or failover drill

WHAT TO PRACTICE

Lab 1: Create a Runbook

Define:

- symptoms
- diagnostic steps
- remediation steps
- verification steps

Lab 2: Build an escalation tree

Define L1 → L2 → L3 mapping.

Lab 3: Failover Simulation

Simulate a workload outage → follow your runbook.

WHERE TO STUDY

- Google SRE Workbook
- Incident Response Playbooks
- PagerDuty postmortem examples

INFRATHrone MINDSET NOTES

- A runbook is worthless unless tested.
- Escalation must follow *path of least confusion*.
- DR is successful only when: **“The team can execute the failover half-asleep at 3 AM.”**